Agile Model in Software Development

1. Introduction

In software development, customer requirements and business needs can change frequently. New features may be requested, existing functionalities may need improvements, and issues may be discovered during development.

Traditional development approaches that require all requirements to be finalized at the beginning may become difficult to manage in such situations.

To address these challenges, software teams use the **Agile Model**, which emphasizes iterative development, continuous feedback, collaboration, and the ability to adapt to changing requirements.

Instead of developing the entire application in one long cycle, Agile teams divide the work into smaller, manageable iterations and deliver software incrementally.

2. Definition of Agile Model

The **Agile Model** is an iterative and incremental approach to software development in which a project is divided into smaller development cycles, allowing teams to build, test, review, and deliver working software continuously.

It emphasizes customer collaboration, team communication, frequent delivery, continuous feedback, and adaptability to changing requirements.

The primary objective of Agile is to deliver valuable software frequently while responding effectively to customer needs and changes in business requirements.

3. Why Do We Need Agile?

Consider an online shopping application.

Suppose a company initially plans to develop user registration, login, product browsing, and cart management.

During development, the customer requests additional features such as product recommendations, multiple payment methods, and order tracking.

In a strictly sequential development approach, accommodating these changes may require revisiting previously completed phases.

Agile provides a more flexible approach by allowing teams to prioritize requirements and incorporate changes into upcoming development iterations.

Its major purposes include:

- Delivering working software frequently.
- Adapting to changing business requirements.
- Encouraging continuous customer feedback.
- Improving collaboration between developers and testers.
- Identifying issues earlier through frequent testing.
- Reducing the risk of delivering software that does not meet customer expectations.
- Improving product quality through continuous improvement.

4. Core Principles of Agile

Agile is based on the Agile Manifesto, which defines four important values.

4.1 Individuals and Interactions Over Processes and Tools

Agile emphasizes communication and collaboration between people rather than depending entirely on processes and tools.

Example:

Instead of relying only on Jira comments to resolve a requirement clarification, developers, testers, and the Business Analyst discuss the requirement directly.

4.2 Working Software Over Comprehensive Documentation

Documentation is important, but Agile gives greater emphasis to delivering software that actually works.

The goal is to produce useful, working software rather than spending excessive time preparing documentation that does not directly contribute to product delivery.

Example:

A team develops and demonstrates a working login feature instead of waiting until every application feature has been completed.

4.3 Customer Collaboration Over Contract Negotiation

Agile encourages continuous communication with customers and stakeholders to understand their expectations and obtain feedback.

Example:

After demonstrating a product search feature, the customer suggests adding category-based filtering. The team discusses the request and considers it for an upcoming iteration.

4.4 Responding to Change Over Following a Plan

Agile accepts that requirements can change and encourages teams to adapt their plans when necessary.

Example:

If the customer changes the expected checkout behavior, the team can analyze the impact and update the upcoming development work accordingly.

Important: The Agile Manifesto does not say that processes, documentation, contracts, or plans are unnecessary. It values the items on the left more highly while recognizing the value of those on the right.

5. How Does the Agile Model Work?

Agile divides development work into smaller iterations. Each iteration focuses on delivering a usable increment of the product.

The general process works as follows:

1. Requirements are collected and organized.
2. Work is prioritized based on business value.
3. The team selects work for an iteration.
4. Developers implement the selected functionality.
5. Testers verify the implemented features.
6. The team reviews the completed work.
7. Feedback is collected and improvements are identified.

8. The next iteration begins with updated priorities.

This process continues until the product objectives are achieved.

6. Agile Workflow

Product Requirements

|

v

Product Backlog

|

v

Iteration Planning

|

v

Development and Testing

|

v

Working Increment

|

v

Review / Demo

|

v

Feedback and Improvement

|

v

Next Iteration

|

+----------> Repeat

The iterative approach allows teams to continuously improve the product based on feedback and changing requirements.

7. Important Agile Terminology

7.1 Product Backlog

The Product Backlog is an ordered list of work items required to improve or develop a product.

It may contain:

- New features.

- User Stories.
- Bug fixes.
- Technical improvements.
- Research activities.

Example:

An online shopping application's Product Backlog may contain:

- User registration.
- User login.
- Product search.
- Product filtering.
- Shopping cart.
- Payment integration.
- Order tracking.

7.2 User Story

A User Story is a short description of a functionality from the perspective of the user who needs it.

It commonly follows this format:

"As a [user], I want [functionality] so that [benefit]."

Example:

"As a customer, I want to search for products by name so that I can quickly find the products I need."

User Stories help teams understand functionality from the user's perspective.

7.3 Sprint

A Sprint is a fixed-length period during which a Scrum team works toward a defined goal and creates a usable product increment.

In Scrum, a Sprint is one month or less.

Many teams use one-week, two-week, or three-week Sprints.

Example:

A team may plan a two-week Sprint to develop and test user registration, login, and password reset functionality.

7.4 Product Increment

A Product Increment is the usable outcome produced by completing work during a Sprint.

It combines completed work with previously completed increments and must satisfy the applicable quality requirements.

Example:

After a Sprint, the application may contain fully implemented and tested registration and login functionality.

7.5 Story Points

Story Points are a relative estimation measure used by teams to estimate the effort, complexity, and uncertainty involved in completing a User Story.

They are not direct measurements of hours or days.

Example:

User Story	Story Points
User Login	2
Product Search	3
Payment Integration	8

The team uses these estimates to support Sprint planning and workload discussions.

7.6 Product Owner

The Product Owner is accountable for maximizing product value and managing the Product Backlog effectively.

Responsibilities include:

- Communicating the product vision.
- Ordering Product Backlog items.
- Clarifying requirements.
- Ensuring transparency of backlog items.
- Working with stakeholders.

7.7 Scrum Master

The Scrum Master is accountable for establishing Scrum as defined in the Scrum Guide and improving the team's effectiveness.

Responsibilities include:

- Helping the team understand Scrum.
- Supporting effective Scrum events.
- Helping remove impediments.
- Coaching the team in self-management.
- Supporting continuous improvement.

7.8 Developers

In Scrum, Developers are the people in the Scrum Team who are committed to creating any aspect of a usable Increment each Sprint.

They may include software developers, testers, automation engineers, designers, and other specialists depending on the product and team structure.

8. Scrum Framework in Agile

Scrum is a lightweight framework used to manage complex product development through iterative and incremental work.

It provides defined accountabilities, events, and artifacts that help teams organize their work.

8.1 Scrum Accountabilities

Scrum defines three accountabilities within the Scrum Team:

Accountability	Responsibility
Product Owner	Maximizes product value and manages the Product Backlog.
Scrum Master	Establishes Scrum and improves team effectiveness.
Developers	Create a usable Increment during each Sprint.

The Scrum Team is self-managing and cross-functional.

8.2 Scrum Events

Scrum defines five events, including the Sprint itself.

A. Sprint

The Sprint is the container for all other Scrum events. It is a fixed-length period of one month or less.

B. Sprint Planning

Sprint Planning initiates the Sprint by defining why the Sprint is valuable, what can be done during it, and how the selected work will be accomplished.

The team collaborates to establish the Sprint Goal and select Product Backlog items.

C. Daily Scrum

The Daily Scrum is a 15-minute event for Developers to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as needed.

It is held every working day of the Sprint.

It is not simply a status meeting for managers.

D. Sprint Review

The Sprint Review is held near the end of the Sprint to inspect the outcome of the Sprint and determine future adaptations.

The Scrum Team and stakeholders collaborate to review the product increment and discuss what should happen next.

E. Sprint Retrospective

The Sprint Retrospective is an opportunity for the Scrum Team to inspect how the previous Sprint went regarding individuals, interactions, processes, tools, and the Definition of Done.

The team identifies improvements that can increase its effectiveness and quality.

8.3 Scrum Artifacts

Scrum defines three artifacts.

Artifact	Commitment	Purpose
Product Backlog	Product Goal	Ordered list of work needed to improve the product.
Sprint Backlog	Sprint Goal	Selected Product Backlog items and the plan for delivering them.
Increment	Definition of Done	Usable, completed product work produced during a Sprint.

These artifacts provide transparency and help the team understand the current state of the work.

9. Agile Example – Online Shopping Application

Consider a team developing an online shopping application using Scrum.

Instead of developing the entire application before delivering anything, the team divides the work into multiple Sprints.

Sprint	Planned Functionality	Expected Outcome
Sprint 1	User registration and login	Customers can create accounts and log in.
Sprint 2	Product listing and search	Customers can browse and search for products.
Sprint 3	Shopping cart and coupon functionality	Customers can manage cart items and apply eligible coupons.
Sprint 4	Checkout and payment	Customers can complete orders and receive confirmation.

At the end of each Sprint, the team reviews the completed work, gathers feedback, and plans the next Sprint.

The application gradually becomes more complete through usable increments.

10. Role of Testing in Agile

Testing is not postponed until the end of the entire project in Agile.

Testing activities take place continuously throughout development.

Developers and testers collaborate to verify functionality as it is implemented.

Testing activities include:

- Reviewing User Stories and acceptance criteria.
- Identifying test scenarios during requirement discussions.
- Preparing test cases for upcoming work.
- Performing testing during the Sprint.
- Reporting and tracking defects.
- Retesting fixes.
- Performing regression testing.
- Automating suitable test cases.
- Supporting continuous integration and delivery.

Example:

Suppose developers complete the product search functionality during a Sprint.

The tester does not wait until all application modules are finished. The tester verifies keyword search, case sensitivity, unmatched search terms, and expected results as soon as the functionality becomes available.

This allows defects to be identified and addressed earlier.

11. Agile Tools

Agile teams use project management, collaboration, testing, and development tools to manage their work.

Tool	Common Purpose
Jira	Managing Backlogs, User Stories, Sprints, and Bugs.
Confluence	Maintaining project documentation and team knowledge.

Xray	Managing Test Cases, Test Plans, and Test Executions within Jira.
Git	Version control and source code collaboration.
Jenkins	Automating build and CI/CD processes.
Selenium	Automating web application testing.

The tools selected depend on the organization's technology stack and working practices.

12. Advantages of the Agile Model

- **Flexibility:** Allows teams to adapt to changing requirements.
- **Frequent Delivery:** Working software is delivered incrementally.
- **Continuous Feedback:** Stakeholders can review progress regularly.
- **Early Defect Detection:** Continuous testing helps identify issues earlier.
- **Better Collaboration:** Encourages communication between developers, testers, and stakeholders.
- **Continuous Improvement:** Retrospectives help teams improve their processes.
- **Improved Visibility:** Backlogs, Sprint Boards, and reviews make progress easier to monitor.

13. Disadvantages and Challenges of Agile

- **Requires Continuous Collaboration:** Teams need regular communication and stakeholder involvement.
- **Changing Requirements:** Frequent changes can create planning challenges if not managed properly.
- **Less Predictable Long-Term Scope:** The final scope and schedule may evolve as priorities change.
- **Requires Active Stakeholder Participation:** Delayed feedback can affect progress.
- **Documentation Risks:** Teams may create insufficient documentation if they misunderstand Agile's emphasis on working software.
- **Coordination Challenges:** Large or distributed teams may need additional coordination.

14. When Should We Use Agile?

Agile is particularly useful when:

- Requirements may change during development.
- Customer feedback is important.
- Software needs to be delivered incrementally.
- The product requires continuous improvements.
- Teams can collaborate frequently.
- Early delivery of valuable functionality is beneficial.

It is commonly used in web applications, mobile applications, SaaS products, and continuously evolving software systems.

15. Conclusion

The Agile Model is an iterative and incremental approach to software development that emphasizes collaboration, continuous feedback, frequent delivery, and adaptability.

It enables teams to develop software in smaller increments rather than waiting until the entire application is completed.

Frameworks such as Scrum provide structured ways to apply Agile principles through Sprints, accountabilities, events, and artifacts.

Understanding Agile is essential for software developers, testers, and SDETs because modern software teams frequently use Agile practices to plan, develop, test, and deliver software.